2026 Vision Training Recipes and Scaling Laws 연구노트

기준일: 2026-04-22

목적

이 문서는 좋은 비전 모델은 아키텍처만으로 결정되지 않는다는 전제 아래, 최근 비전 연구에서 성능 차이를 실제로 만들어 온 학습 recipe와 scaling law를 정리한 노트다.

여기서 말하는 recipe는 단순한 하이퍼파라미터 목록이 아니다.
오히려 아래를 함께 포함하는 훈련 설계 전체를 뜻한다.

- optimizer와 learning rate schedule
- augmentation과 regularization
- input resolution과 cropping policy
- distillation, self-supervised pretraining, transfer setup
- model/data/compute scaling 전략

핵심 질문은 아래다.

- 왜 ViT와 CNN 비교는 recipe를 통제하지 않으면 거의 의미가 없어지는가
- 2026년 기준 비전 scaling law는 어느 정도까지 믿을 만한가
- CNN 기반 영상처리 엔지니어가 실전에 가져갈 recipe 관점은 무엇인가

빠른 결론

1. 최근 비전 모델의 성능 차이 상당수는 구조 그 자체보다 학습 recipe 차이에서 난다.
2. ViT의 약점으로 보였던 많은 부분은 아키텍처 본질만이 아니라, pretraining·AugReg·optimizer 설계 부족에서 왔다.
3. ConvNet도 recipe를 현대화하면 매우 강해지며, ConvNeXt 계열은 그 대표 사례다.
4. vision scaling law는 분명히 존재하지만, NLP보다 데이터 품질·해상도·태스크 구조의 영향을 더 크게 받는다.
5. 실전에서 중요한 것은 무작정 큰 모델이 아니라, 주어진 compute와 data regime에서 가장 잘 맞는 recipe를 찾는 것이다.

---

1. 왜 지금 training recipe를 따로 정리해야 하나

예전에는 모델 논문이 곧 성능 차이의 거의 전부인 것처럼 보였다.

- 더 좋은 block
- 더 좋은 attention
- 더 좋은 neck
- 더 좋은 loss

하지만 최근 비전에서는 같은 구조라도 아래에 따라 결과가 크게 바뀐다.

- AdamW를 쓰는가 SGD를 쓰는가
- Mixup/CutMix/RandAugment를 어디까지 넣는가

- stochastic depth를 얼마나 쓰는가
- EMA를 쓰는가
- warmup과 cosine decay를 쓰는가
- pretraining을 supervised로 했는가 self-supervised로 했는가
- 해상도와 patch size를 어떻게 맞추는가

즉 지금의 모델 비교는 대개 `architecture comparison`이 아니라 `architecture + recipe bundle comparison`이다.

이걸 분리해서 보지 않으면 아래 같은 오해가 생긴다.

- ViT는 데이터가 너무 많이 필요하다
- CNN은 이제 한계다
- 큰 모델이면 무조건 잘 된다

실제로는 이 셋 다 절반만 맞는 말이다.

2. recipe는 무엇으로 구성되나

실전에서 recipe를 볼 때는 적어도 다섯 층으로 나누는 편이 좋다.

2-1. optimizer와 schedule

- SGD vs AdamW
- warmup 길이
- cosine decay 여부
- gradient clipping
- EMA 사용 여부

ViT 계열은 대체로 AdamW와 긴 warmup, 적절한 weight decay에 더 민감하다.
반면 많은 CNN 계열은 SGD 기반에서도 여전히 강하지만, 최근 ConvNet은 AdamW 기반 recipe와 더 잘 결합되기도 한다.[1][2]

2-2. augmentation과 regularization

- RandAugment
- Mixup
- CutMix
- label smoothing
- dropout
- stochastic depth
- repeated augmentation

`How to train your ViT?`는 ViT의 데이터 효율을 이해하려면 AugReg를 함께 봐야 한다는 점을 명확히 보여 줬다.[3]

2-3. resolution과 patch/interface 설계

- train/test resolution mismatch
- patch size

- resize와 crop policy
- dense task 전이 시 multi-scale compatibility

이미지 분류에서는 괜찮던 설정이 detection/segmentation으로 가면 갑자기 비효율적이 되는 경우가 많다.

2-4. teacher signal과 initialization

- supervised pretraining
- self-supervised pretraining
- distillation
- synthetic pretraining
- modality-specific pretraining

`DeiT`는 distillation과 data-efficient training recipe가 ViT 실용화를 크게 앞당긴 대표 작업이다.[4]

2-5. scaling 전략

- 모델 크기만 키우는가
- 데이터도 같이 키우는가
- 해상도도 같이 키우는가
- compute budget 안에서 depth/width/token 수를 어떻게 맞추는가

좋은 recipe는 결국 `학습 자원 배분 전략`이다.

3. ViT가 강해진 건 구조만이 아니었다

ViT 초기 서사는 대략 이랬다.

- 구조는 단순하다
- 하지만 큰 데이터와 큰 compute가 필요하다
- scratch training은 불리하다

이 자체는 대체로 맞았다.
원래 ViT는 대규모 pretraining과 함께 강해진 모델이었다.[5]

하지만 그 뒤 흐름이 중요하다.

3-1. DeiT: recipe가 ViT를 실전 모델로 바꿨다

`Training data-efficient image transformers & distillation through attention`은 ViT를 보다 적은 데이터 체계에서도 경쟁력 있게 만들 수 있음을 보여 줬다.[4]

핵심은 아래였다.

- teacher distillation
- 적절한 augmentation
- data-efficient supervised training

이 작업 이후 비전 쪽에서 “ViT는 데이터가 너무 많이 들어서 실전성이 없다”는 평가는 훨씬 약해졌다.

3-2. AugReg 연구: 데이터 부족을 recipe로 일부 상쇄할 수 있다

`How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers`는 AugReg와 compute가 데이터 부족을 어느 정도 상쇄할 수 있음을 보여 줬다.[3]

이 논문이 중요한 이유는 다음과 같다.

- ViT의 약점은 부분적으로 recipe sensitivity다
- 충분한 AugReg가 있으면 public-scale data로도 강한 전이 성능을 얻을 수 있다
- 모델 크기, 데이터 크기, AugReg, compute는 함께 봐야 한다

즉 `아키텍처가 약해서` 성능이 낮은 것과 `recipe가 맞지 않아서` 낮은 것을 구분해야 한다.

3-3. scratch regime 재평가

`When Vision Transformers Outperform ResNets without Pre-training or Strong Data Augmentations`는 optimizer와 loss geometry 관점에서 scratch ViT를 다시 보게 만든 작업이다.[6]

또 `Bootstrapping ViTs`는 대규모 pretraining 없이도 ViT를 더 경쟁력 있게 학습시키는 절차를 제안했다.[7]

이 흐름이 주는 교훈은 간단하다.

- ViT가 무조건 pretraining 의존적이라고 단정하면 틀릴 수 있다
- 하지만 그 대가로 optimization 설계가 훨씬 더 중요해진다

4. ConvNet도 recipe를 바꾸면 다시 강해진다

많은 사람이 Transformer의 등장을 `CNN 시대 종료`로 해석했다.
하지만 실제로는 CNN도 recipe 현대화의 혜택을 크게 받았다.

4-1. ConvNeXt: 아키텍처와 recipe의 공동 재설계

`A ConvNet for the 2020s`는 ResNet 계열을 Transformer 시대의 훈련 관행에 맞게 재설계하면 pure ConvNet도 매우 강할 수 있음을 보여 줬다.[1]

중요한 포인트는 아래다.

- patchify stem
- large kernel
- inverted bottleneck 류 설계
- LayerNorm 사용
- AdamW 등 현대적 optimization

이건 단지 block 하나를 바꾼 게 아니라 `ConvNet을 학습시키는 문법` 자체를 바꾼 것이다.

4-2. ConvNeXt V2: self-supervised recipe co-design

`ConvNeXt V2`는 masked autoencoder류 pretraining을 ConvNet 쪽으로 끌어오며, backbone과 self-supervised recipe의 공동 설계를 보여 줬다.[2]

이 작업의 함의는 명확하다.

- 좋은 self-supervised recipe는 Transformer 전용 자산이 아니다
- ConvNet도 적절한 pretext와 normalization 설계를 통해 scaling 이득을 받을 수 있다

- backbone 비교는 pretraining recipe를 고정하지 않으면 공정하지 않다

5. self-supervised pretraining은 이제 recipe의 핵심 축이다

최근 비전에서 pretraining은 옵션이 아니라 baseline에 가까워졌다.

- supervised pretraining
- contrastive SSL
- masked image modeling
- distillation-based SSL

그중 `Masked Autoencoders Are Scalable Vision Learners`는 masked modeling이 scaling-friendly한 비전 pretraining recipe임을 강하게 보여 준 대표 논문이다.[8]

MAE가 중요했던 이유는 아래다.

- 비대칭 encoder-decoder로 효율적이다
- 높은 masking ratio가 가능하다
- 대형 ViT 학습을 더 실용적으로 만든다

실전적으로 보면, 이제 backbone 논의는 아래로 바뀌었다.

- 어떤 구조인가
- 어떤 pretraining recipe를 썼는가
- fine-tuning protocol은 무엇인가

6. vision scaling law는 어떻게 봐야 하나

NLP에서는 scaling law가 거의 하나의 언어처럼 쓰인다.
비전도 비슷한 방향으로 갔지만, 해석은 더 조심해야 한다.

6-1. 비전에서도 scale는 분명히 핵심이다

`Scaling Vision Transformers`는 모델, 데이터, compute를 키울 때 ViT가 어떻게 반응하는지 체계적으로 보여 준 대표 작업이다.[9]

이 논문 이후 명확해진 점은 아래다.

- 비전에서도 규모는 분명히 중요한 축이다
- 작은 모델의 상대 비교만으로 미래 우위를 단정하면 위험하다
- 데이터와 compute를 함께 봐야 한다

6-2. 하지만 비전은 "무조건 더 크게"가 잘 통하지 않는다

`Getting ViT in Shape`는 같은 compute budget 안에서도 model shape를 잘 고르는 것이 중요함을 보여 줬다.[10]

즉 scaling law의 실전 해석은 아래에 가깝다.

- 더 큰 모델이 항상 더 좋은 게 아니다
- 같은 FLOPs에서도 depth/width/patch 구성 차이가 크다
- compute-optimal design이 중요하다

이 지점은 CNN 엔지니어에게도 중요하다.
큰 backbone 하나로 밀어붙이기보다 문제 해상도와 데이터 크기에 맞는 구조 형태를 고르는 것이 훨씬 효율적일 수 있다.

6-3. 해상도 scaling은 vision에서 특히 중요하다

비전 모델은 token 수가 곧 resolution과 강하게 연결된다.
그래서 해상도를 올릴 때 드는 비용과 이득을 별도로 봐야 한다.

`Swin Transformer V2`는 capacity와 resolution scaling을 동시에 다룬 대표 사례다.[11]

이 흐름은 dense task에서 특히 중요하다.

- classification에서의 최적 patch size
- detection에서의 feature map 유지 비용
- segmentation에서의 decoder cost
- 실제 wall-clock latency

즉 vision scaling law는 `parameter count`보다 `resolution-aware systems law`에 더 가깝다.

7. recipe와 scaling은 태스크에 따라 의미가 달라진다

같은 backbone이라도 아래 태스크에서는 recipe 중요도가 다르게 나타난다.

7-1. image classification

- AugReg 효과가 직접적으로 잘 보인다
- distillation 이득이 크다
- large-scale pretraining의 효과가 비교적 명확하다

7-2. detection / segmentation

- backbone top-1이 그대로 전이되지 않는다
- multi-scale compatibility와 dense feature quality가 더 중요하다
- fine-tuning schedule과 neck 결합 방식의 영향이 크다

7-3. small-data / domain shift

- scale보다 regularization stability가 더 중요할 수 있다
- calibration이 accuracy보다 실전 가치가 클 수 있다
- pretraining source와 target mismatch가 커지면 scaling 이득이 약해질 수 있다

즉 좋은 scaling rule은 보편 법칙이라기보다 문제 구조에 조건부인 경험 법칙으로 보는 편이 안전하다.

8. CNN 기반 영상처리 엔지니어를 위한 실전 해석

실전에서 가장 흔한 실수는 아래 두 가지다.

1. backbone만 바꾸고 recipe는 예전 그대로 두는 것
2. 논문 기본값을 그대로 가져오고 자기 데이터 조건을 무시하는 것

현실적인 프레임은 아래가 더 낫다.

8-1. 먼저 data regime를 정의한다

- 샘플 수가 충분한가
- 라벨 품질이 안정적인가
- 해상도가 높은가
- domain shift가 큰가

8-2. 그다음 backbone보다 recipe를 먼저 고친다

- optimizer
- schedule
- augmentation
- regularization
- fine-tuning resolution

특히 small-data 환경에서는 backbone 교체보다 recipe 정리가 더 큰 이득일 때가 많다.

8-3. scaling은 반드시 wall-clock 기준으로 본다

- GPU 메모리
- training time
- inference latency
- 실제 입력 해상도

논문상 FLOPs 이득이 실전 throughput 이득으로 이어지지 않는 경우는 매우 흔하다.

8-4. 비교 실험은 반드시 bundle 단위로 기록한다

- backbone
- pretraining
- optimizer
- augmentation
- resolution
- schedule

이 여섯 개를 한 묶음으로 남기지 않으면 나중에 결과를 해석할 수 없다.

9. 바로 해볼 만한 실험 질문

이 주제로 실제 프로젝트를 돌린다면 아래 질문들이 값어치가 크다.

1. ViT와 ConvNeXt 차이의 몇 퍼센트가 backbone 차이이고, 몇 퍼센트가 recipe 차이인가
2. small-data에서 AdamW + 강한 AugReg가 SGD baseline을 실제로 얼마나 넘는가
3. 동일 compute에서 width/depth/resolution trade-off의 최적점은 어디인가
4. MAE pretraining 이득은 detection과 segmentation에서 각각 얼마나 오래 남는가
5. clean accuracy를 조금 포기하면 calibration과 robustness를 얼마나 얻을 수 있는가

10. 읽기 우선순위

이 주제를 빠르게 잡으려면 아래 순서가 좋다.

1. ViT, DeiT
   ViT가 왜 강했는지, 왜 recipe가 중요했는지 파악
2. How to train your ViT, scratch ViT 계열
   AugReg와 optimizer 영향 이해
3. ConvNeXt, ConvNeXt V2
   ConvNet 현대화와 recipe 공동 설계 이해
4. MAE
   self-supervised scaling 축 이해
5. Scaling Vision Transformers, Getting ViT in Shape, Swin V2
   vision scaling law를 보다 현실적으로 이해

최종 정리

2026년 기준으로 비전 모델 연구를 구조 비교만으로 읽는 것은 거의 항상 불완전하다.

더 정확한 프레임은 아래다.

- backbone은 recipe와 함께 해석해야 한다
- scale는 중요하지만, data quality와 resolution cost를 무시하면 착시가 생긴다
- ViT의 강함도, ConvNet의 부활도 상당 부분은 학습 설계 덕분이다
- 실전 경쟁력은 "가장 큰 모델"이 아니라 "내 문제에 맞는 가장 정교한 recipe"에서 나온다

즉 좋은 비전 엔지니어링은 모델 선택보다 한 단계 위인 훈련 시스템 설계에 가깝다.

참고문헌

[1] Liu et al., *A ConvNet for the 2020s* (CVPR 2022)
CVF Open Access

[2] Woo et al., *ConvNeXt V2: Co-Designing and Scaling ConvNets With Masked Autoencoders* (CVPR 2023)
CVF Open Access

[3] Steiner et al., *How to train your ViT? Data, Augmentation, and Regularization in Vision Transformers* (2021)
arXiv

[4] Touvron et al., *Training data-efficient image transformers & distillation through attention* (ICML 2021)
PMLR

[5] Dosovitskiy et al., *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* (ICLR 2021)
OpenReview

[6] Chen et al., *When Vision Transformers Outperform ResNets without Pre-training or Strong Data Augmentations* (ICLR 2022)
arXiv

[7] Zhang et al., *Bootstrapping ViTs: Towards Liberating Vision Transformers from Pre-training* (CVPR 2022)
CVF Open Access PDF

[8] He et al., *Masked Autoencoders Are Scalable Vision Learners* (CVPR 2022)
CVF Open Access

[9] Zhai et al., *Scaling Vision Transformers* (2021)
arXiv

[10] Alabdulmohsin et al., *Getting ViT in Shape: Scaling Laws for Compute-Optimal Model Design* (2023)
arXiv

[11] Liu et al., *Swin Transformer V2: Scaling Up Capacity and Resolution* (CVPR 2022)
arXiv